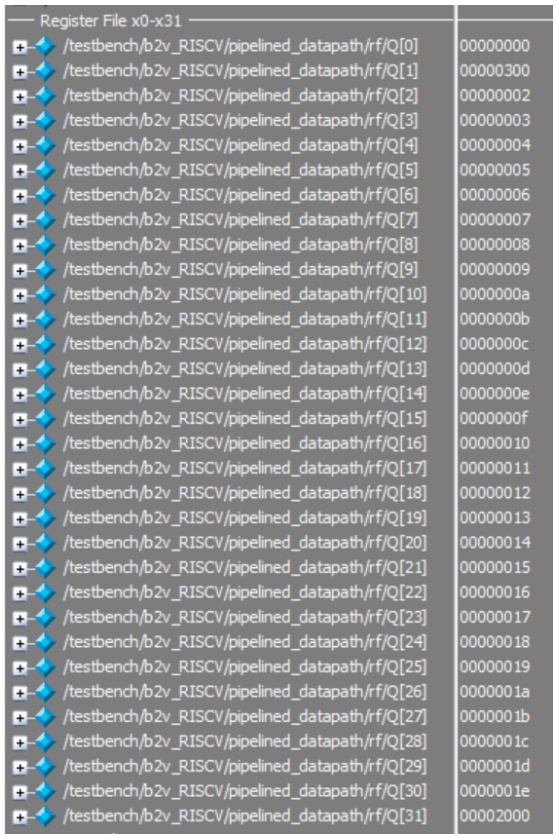
Overall CPU Test: To verify my pipelined RISC-V processor, I used the same general assembly code form Learning Activity 5 and 6 to test the CPU and memory. Correct functionality is verified by comparing the final register contents and RAM contents from the RARS simulator with the results from my pipelined CPU in ModelSim. The main CPU test is used to verify overall correctness, while the hazard tests are used to specifically verify the new pipelined behavior.

RARS Simulator Register Contents vs. Register Contents of the Pipelined CPU:

Registers | Floating Point | Control and Status

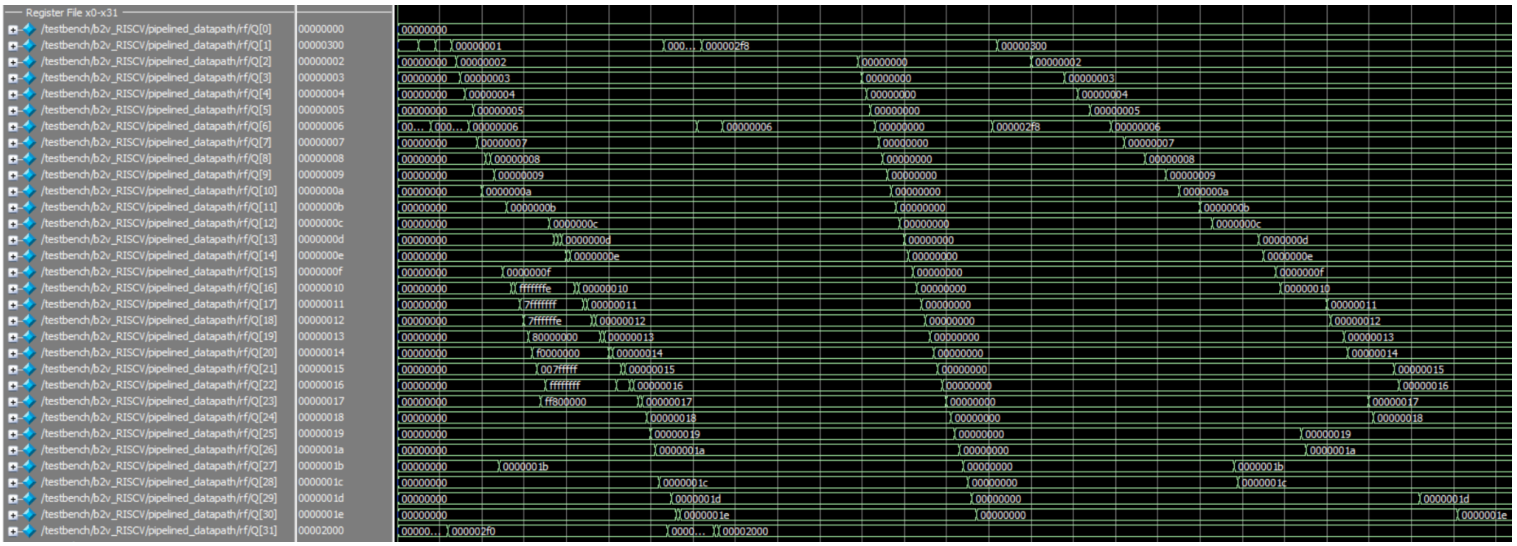
Name	Number	Value
zero	0	0x00000000
ra	1	0x00000300
sp	2	0x00000002
gp	3	0x00000003
tp	4	0x00000004
t0	5	0x00000005
t1	6	0x00000006
t2	7	0x00000007
s0	8	0x00000008
s1	9	0x00000009
a0	10	0x0000000a
a1	11	0x0000000b
a2	12	0x0000000c
a3	13	0x0000000d
a4	14	0x0000000e
a5	15	0x0000000f
a6	16	0x00000010
a7	17	0x00000011
s2	18	0x00000012
s3	19	0x00000013
s4	20	0x00000014
s5	21	0x00000015
s6	22	0x00000016
s7	23	0x00000017
s8	24	0x00000018
s9	25	0x00000019
s10	26	0x0000001a
s11	27	0x0000001b
t3	28	0x0000001c
t4	29	0x0000001d
t5	30	0x0000001e
t6	31	0x00002000
pc		0x00000300

RARS Simulator



Pipelined CPU

Full Wave Form for Pipelined CPU Register Contents:



RARS Simulator RAM Contents vs. RAM Contents of the Pipelined CPU:

Data Segment

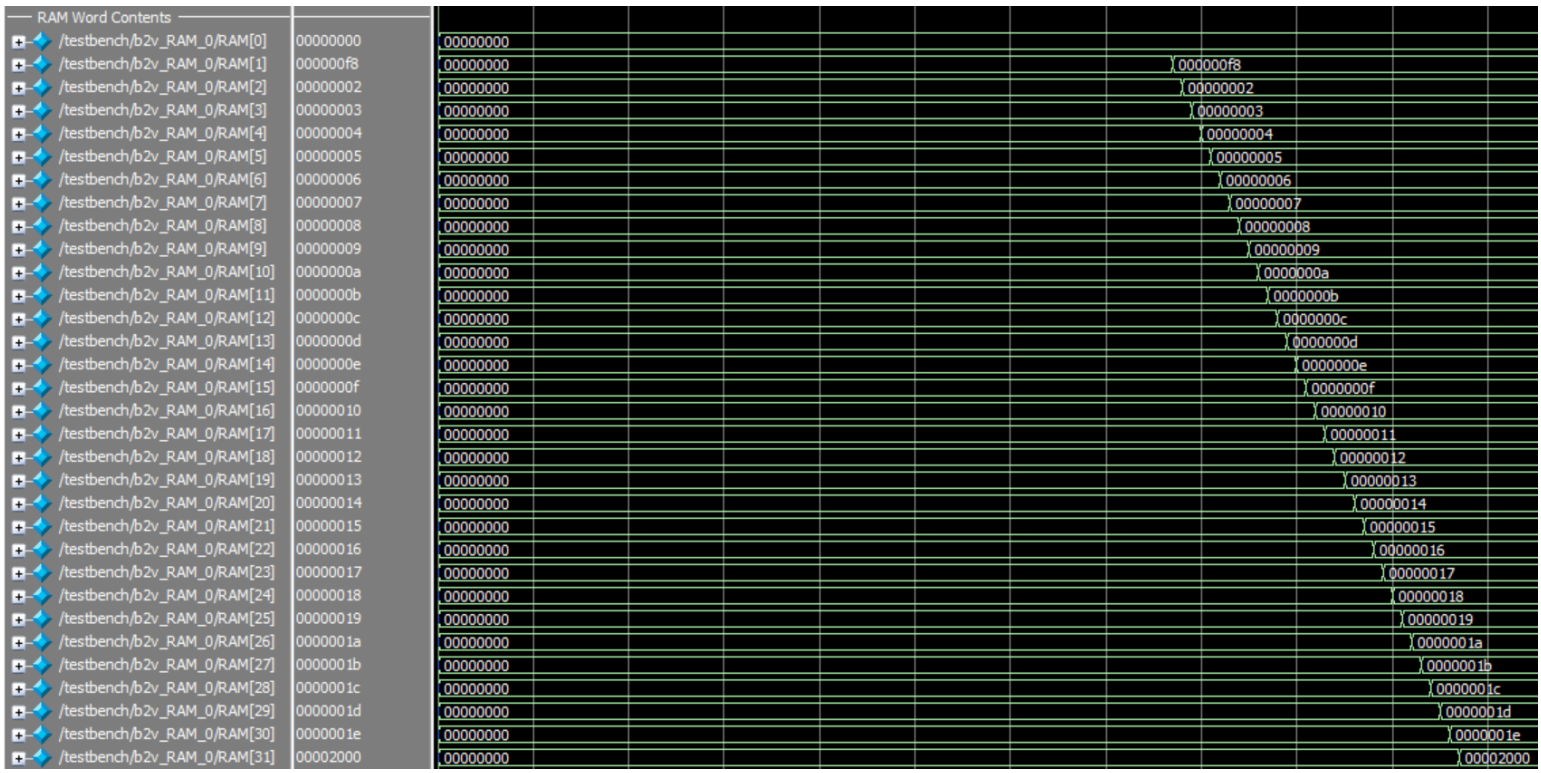
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x00002000	0x00000000	0x000000f8	0x00000002	0x00000003	0x00000004	0x00000005	0x00000006	0x00000007
0x00002020	0x00000008	0x00000009	0x0000000a	0x0000000b	0x0000000c	0x0000000d	0x0000000e	0x0000000f
0x00002040	0x00000010	0x00000011	0x00000012	0x00000013	0x00000014	0x00000015	0x00000016	0x00000017
0x00002060	0x00000018	0x00000019	0x0000001a	0x0000001b	0x0000001c	0x0000001d	0x0000001e	0x00002000

RARS Simulator RAM Contents

RAM Word Contents	
/testbench/b2v_RAM_0/RAM[0]	00000000
/testbench/b2v_RAM_0/RAM[1]	000000f8
/testbench/b2v_RAM_0/RAM[2]	00000002
/testbench/b2v_RAM_0/RAM[3]	00000003
/testbench/b2v_RAM_0/RAM[4]	00000004
/testbench/b2v_RAM_0/RAM[5]	00000005
/testbench/b2v_RAM_0/RAM[6]	00000006
/testbench/b2v_RAM_0/RAM[7]	00000007
/testbench/b2v_RAM_0/RAM[8]	00000008
/testbench/b2v_RAM_0/RAM[9]	00000009
/testbench/b2v_RAM_0/RAM[10]	0000000a
/testbench/b2v_RAM_0/RAM[11]	0000000b
/testbench/b2v_RAM_0/RAM[12]	0000000c
/testbench/b2v_RAM_0/RAM[13]	0000000d
/testbench/b2v_RAM_0/RAM[14]	0000000e
/testbench/b2v_RAM_0/RAM[15]	0000000f
/testbench/b2v_RAM_0/RAM[16]	00000010
/testbench/b2v_RAM_0/RAM[17]	00000011
/testbench/b2v_RAM_0/RAM[18]	00000012
/testbench/b2v_RAM_0/RAM[19]	00000013
/testbench/b2v_RAM_0/RAM[20]	00000014
/testbench/b2v_RAM_0/RAM[21]	00000015
/testbench/b2v_RAM_0/RAM[22]	00000016
/testbench/b2v_RAM_0/RAM[23]	00000017
/testbench/b2v_RAM_0/RAM[24]	00000018
/testbench/b2v_RAM_0/RAM[25]	00000019
/testbench/b2v_RAM_0/RAM[26]	0000001a
/testbench/b2v_RAM_0/RAM[27]	0000001b
/testbench/b2v_RAM_0/RAM[28]	0000001c
/testbench/b2v_RAM_0/RAM[29]	0000001d
/testbench/b2v_RAM_0/RAM[30]	0000001e
/testbench/b2v_RAM_0/RAM[31]	00002000

Pipelined CPU RAM Contents

Full wave Form for Pipelined CPU RAM Contents:



Seeing that the final register contents and RAM contents from the ModelSim simulation match the expected values from the RARS simulator, it is evident that the pipelined CPU is functioning correctly. The matching register values show that the CPU correctly executes arithmetic, logic, branch, and writeback operations, while the matching RAM values show that load and store instructions are working properly. Because both the register file and memory contents match the expected results, this verifies that the CPU is fully functional for the tested instruction set.

Hazards Test: To verify that my pipelined CPU correctly handles hazards, I created separate tests for register forwarding, pipeline stall, and pipeline flush.

Register Forwarding Verification:
The figure below shows the register forwarding test. In this test, the add instruction produces the value for register x24, and the following sub instruction immediately needs x24 as an input. This creates a data hazard because x24 has not been written back to the register file yet.

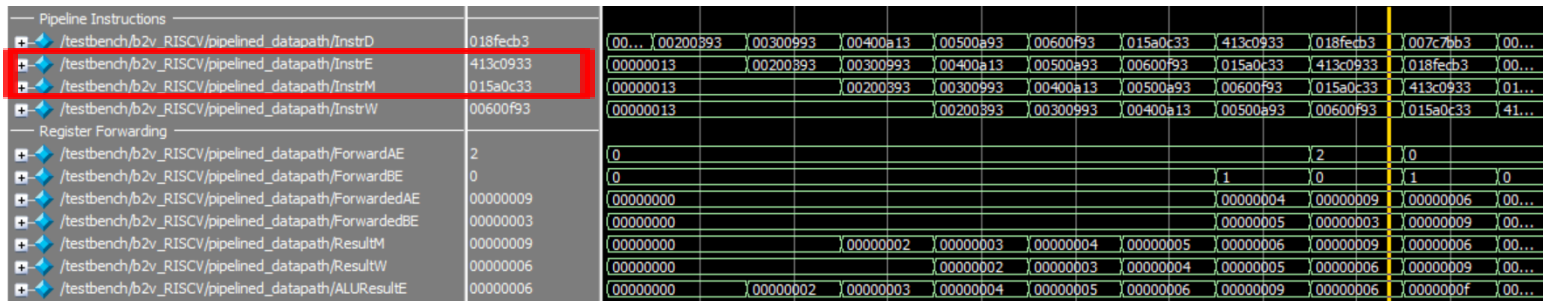
register_forwarding_test.s*

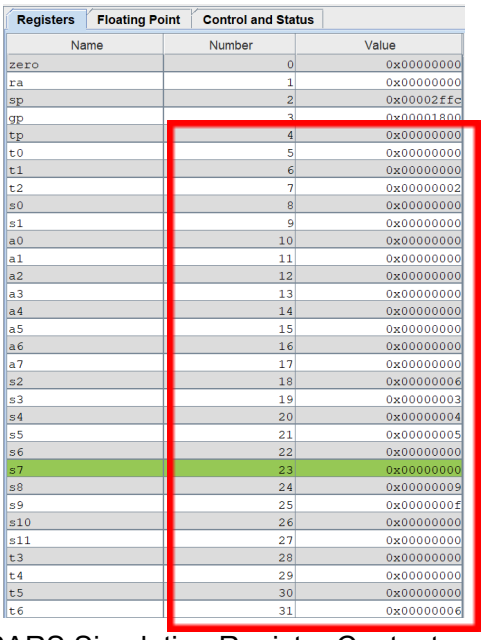
```
1  .section .text.main
2  .globl main
3
4  main:
5      addi t2, x0, 2        # x7  = 2
6      addi s3, x0, 3        # x19 = 3
7      addi s4, x0, 4        # x20 = 4
8      addi s5, x0, 5        # x21 = 5
9      addi t6, x0, 6        # x31 = 6
10
11     add  s8, s4, s5       # x24 = x20 + x21 = 9
12     sub  s2, s8, s3       # x18 = x24 - x19 = 6
13     or   s9, t6, s8       # x25 = x31 OR x24 = 15
14     and  s7, s8, t2       # x23 = x24 AND x7 = 0
15
16 done:
17     beq x0, x0, done      # infinite loop
```

Assembly code for Register Forwarding Test

Code	Basic	Source
0x00200393	addi x7,x0,2	5: addi t2, x0, 2 # x7 = 2
0x00300993	addi x19,x0,3	6: addi s3, x0, 3 # x19 = 3
0x00400a13	addi x20,x0,4	7: addi s4, x0, 4 # x20 = 4
0x00500a93	addi x21,x0,5	8: addi s5, x0, 5 # x21 = 5
0x00600f93	addi x31,x0,6	9: addi t6, x0, 6 # x31 = 6
0x015a0c33	add x24,x20,x21	11: add s8, s4, s5 # x24 = x20 + x21 = 9
0x413c0933	sub x18,x24,x19	12: sub s2, s8, s3 # x18 = x24 - x19 = 6
0x018fecb3	or x25,x31,x24	13: or s9, t6, s8 # x25 = x31 OR x24 = 15
0x007c7bb3	and x23,x24,x7	14: and s7, s8, t2 # x23 = x24 AND x7 = 0
0x00000063	beq x0,x0,0x00000000	17: beq x0, x0, done # infinite loop

In the waveform, the add instruction is in the Memory stage while the sub instruction is in the Execute stage. Since the Execute stage needs the result from the Memory stage, ForwardAE is asserted. This causes ResultM, which contains 0x00000009, to be forwarded into the ALU input. Because of this, the sub instruction can correctly execute without waiting for writeback.

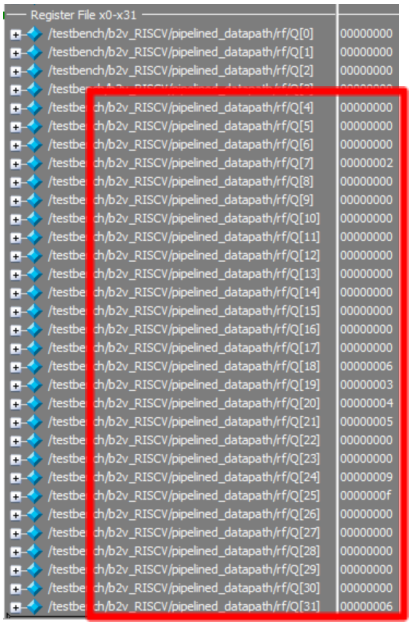




Registers | Floating Point | Control and Status

Name	Number	Value
zero	0	0x00000000
ra	1	0x00000000
sp	2	0x00002ffc
gp	3	0x00001800
tp	4	0x00000000
t0	5	0x00000000
t1	6	0x00000000
t2	7	0x00000002
s0	8	0x00000000
s1	9	0x00000000
a0	10	0x00000000
a1	11	0x00000000
a2	12	0x00000000
a3	13	0x00000000
a4	14	0x00000000
a5	15	0x00000000
a6	16	0x00000000
a7	17	0x00000000
s2	18	0x00000006
s3	19	0x00000003
s4	20	0x00000004
s5	21	0x00000005
s6	22	0x00000000
s7	23	0x00000000
s8	24	0x00000009
s9	25	0x0000000f
s10	26	0x00000000
s11	27	0x00000000
t3	28	0x00000000
t4	29	0x00000000
t5	30	0x00000000
t6	31	0x00000006

RARS Simulation Register Contents



Register File x0-x31

/testbench/b2v_RISCV/pipelined_datapath/rf/Q[0]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[1]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[2]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[3]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[4]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[5]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[6]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[7]	00000002
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[8]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[9]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[10]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[11]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[12]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[13]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[14]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[15]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[16]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[17]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[18]	00000006
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[19]	00000003
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[20]	00000004
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[21]	00000005
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[22]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[23]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[24]	00000009
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[25]	0000000f
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[26]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[27]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[28]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[29]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[30]	00000000
/testbench/b2v_RISCV/pipelined_datapath/rf/Q[31]	00000006

Pipelined CPU Register Contents

At the end of the simulation, the register values match the RARS simulator, proving that register forwarding is working correctly.

Pipeline Stall Verification:
The figure below shows the pipeline stall test. In this test, a lw instruction loads a value into register x23, and the next instruction immediately uses x23. This creates a load-use hazard because the loaded value is not available until the end of the Memory stage.

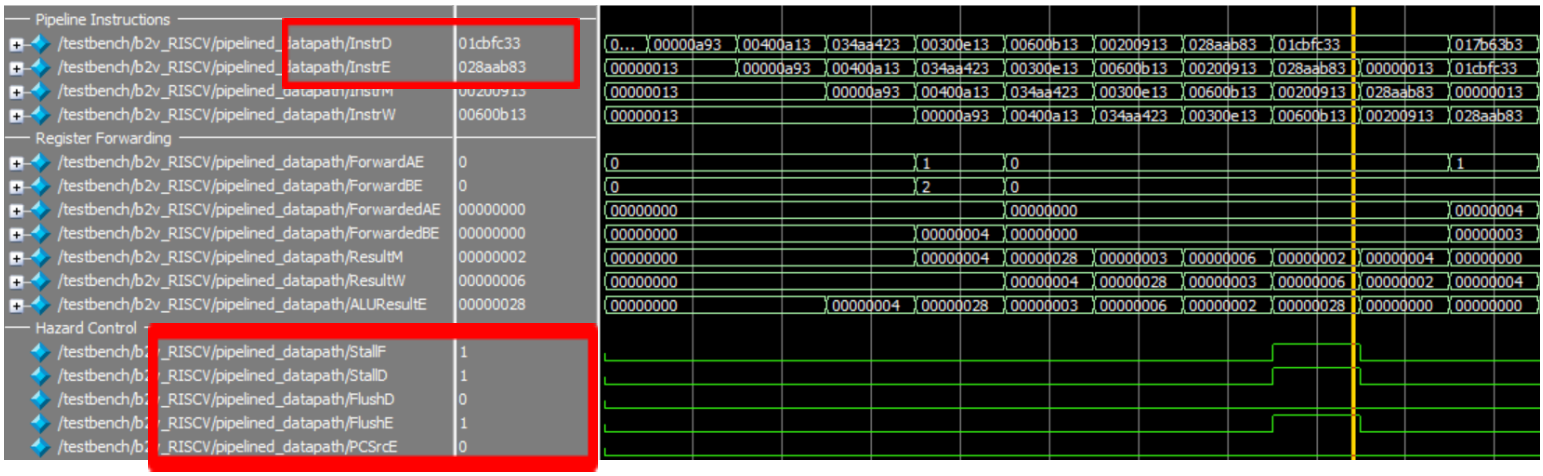
stall_test.s*

```
1  .section .text.main
2  .globl main
3
4  main:
5      addi s5, x0, 0       # x21 = 0
6      addi s4, x0, 4       # x20 = 4
7      sw   s4, 40(s5)      # memory[40] = 4
8
9      addi t3, x0, 3       # x28 = 3
10     addi s6, x0, 6       # x22 = 6
11     addi s2, x0, 2       # x18 = 2
12
13     lw   s7, 40(s5)      # x23 = memory[40] = 4
14     and  s8, s7, t3      # x24 = x23 AND x28 = 0
15     or   t2, s6, s7      # x7  = x22 OR x23 = 6
16     sub  s3, s7, s2      # x19 = x23 - x18 = 2
17
18 done:
19     beq x0, x0, done     # infinite loop
```

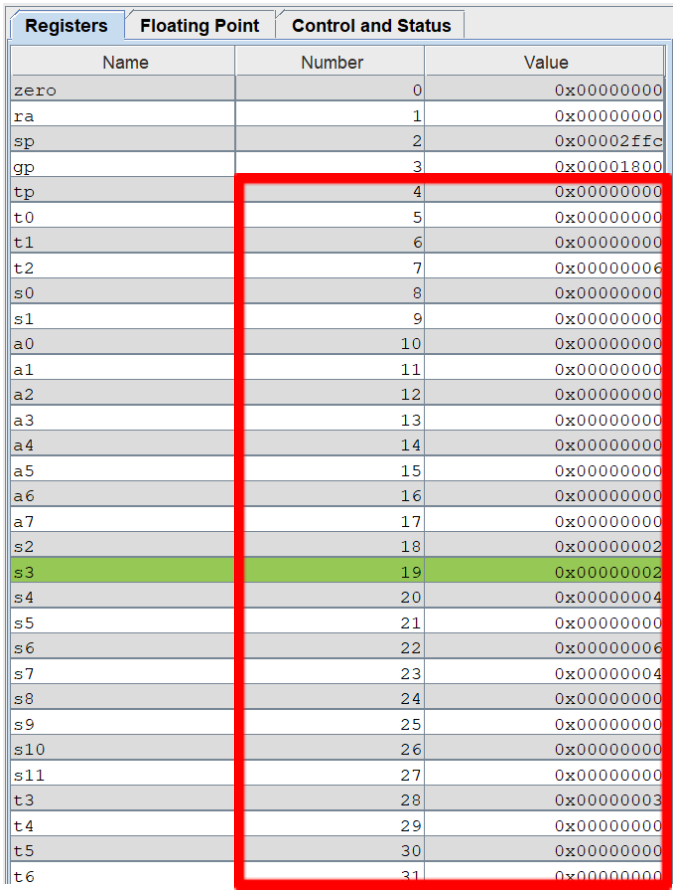
Assembly Code for Pipeline Stall Test

Code	Basic	Source
0x00000a93	addi x21,x0,0	5: addi s5, x0, 0 # x21 = 0
0x00400a13	addi x20,x0,4	6: addi s4, x0, 4 # x20 = 4
0x034aa423	sw x20,0x00000028(x21)	7: sw s4, 40(s5) # memory[40] = 4
0x00300e13	addi x28,x0,3	9: addi t3, x0, 3 # x28 = 3
0x00600b13	addi x22,x0,6	10: addi s6, x0, 6 # x22 = 6
0x00200913	addi x18,x0,2	11: addi s2, x0, 2 # x18 = 2
0x028aab83	lw x23,0x00000028(x21)	13: lw s7, 40(s5) # x23 = memory[40] = 4
0x01cbfc33	and x24,x23,x28	14: and s8, s7, t3 # x24 = x23 AND x28 = 0
0x017b63b3	or x7,x22,x23	15: or t2, s6, s7 # x7 = x22 OR x23 = 6
0x412b89b3	sub x19,x23,x18	16: sub s3, s7, s2 # x19 = x23 - x18 = 2
0x00000063	beq x0,x0,0x00000000	19: beq x0, x0, done # infinite loop

In the waveform, the lw instruction is in the Execute stage while the dependent and instruction is in the Decode stage. Since forwarding cannot provide the value yet, the hazard unit asserts StallF and StallD to freeze the Fetch and Decode stages. FlushE is also asserted, which inserts a NOP instruction into the Execute stage.

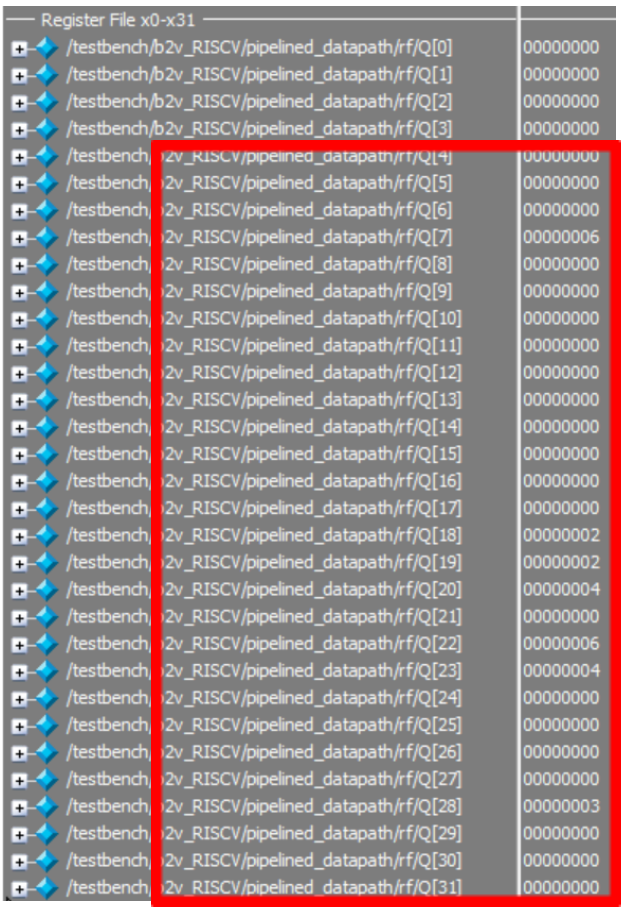


After this one-cycle bubble, the lw instruction reaches the Writeback stage and the loaded value can be forwarded to the dependent instruction. The final register and RAM values match the RARS simulation, proving that the pipeline stall logic is working correctly.

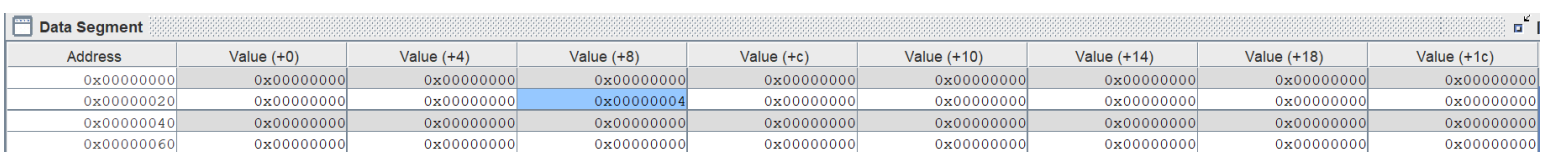


Name	Number	Value
zero	0	0x00000000
ra	1	0x00000000
sp	2	0x00002ffc
gp	3	0x00001800
tp	4	0x00000000
t0	5	0x00000000
t1	6	0x00000000
t2	7	0x00000006
s0	8	0x00000000
s1	9	0x00000000
a0	10	0x00000000
a1	11	0x00000000
a2	12	0x00000000
a3	13	0x00000000
a4	14	0x00000000
a5	15	0x00000000
a6	16	0x00000000
a7	17	0x00000000
s2	18	0x00000002
s3	19	0x00000002
s4	20	0x00000004
s5	21	0x00000000
s6	22	0x00000006
s7	23	0x00000004
s8	24	0x00000000
s9	25	0x00000000
s10	26	0x00000000
s11	27	0x00000000
t3	28	0x00000003
t4	29	0x00000000
t5	30	0x00000000
t6	31	0x00000000

RARS Simulator Register Contents



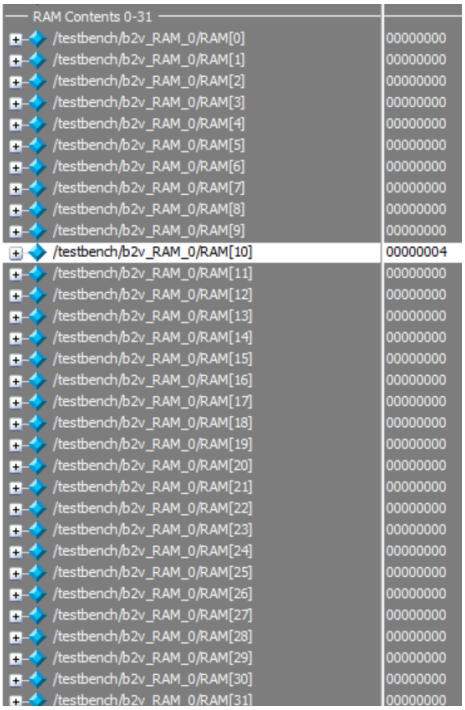
Pipelined CPU Register Contents



Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00000020	0x00000000	0x00000000	0x00000004	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00000040	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00000060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

RARS Simulator RAM Content



Pipelined CPU RAM Content

The final register and RAM values match the RARS simulation, proving that the pipeline stall logic is working correctly.

Pipeline Flush Verification:

The figure below shows the pipeline flush test. In this test, the beq instruction is taken because registers x9 and x18 both contain the value 1. This creates a control hazard because the CPU has already fetched instructions after the branch before the branch decision is known.

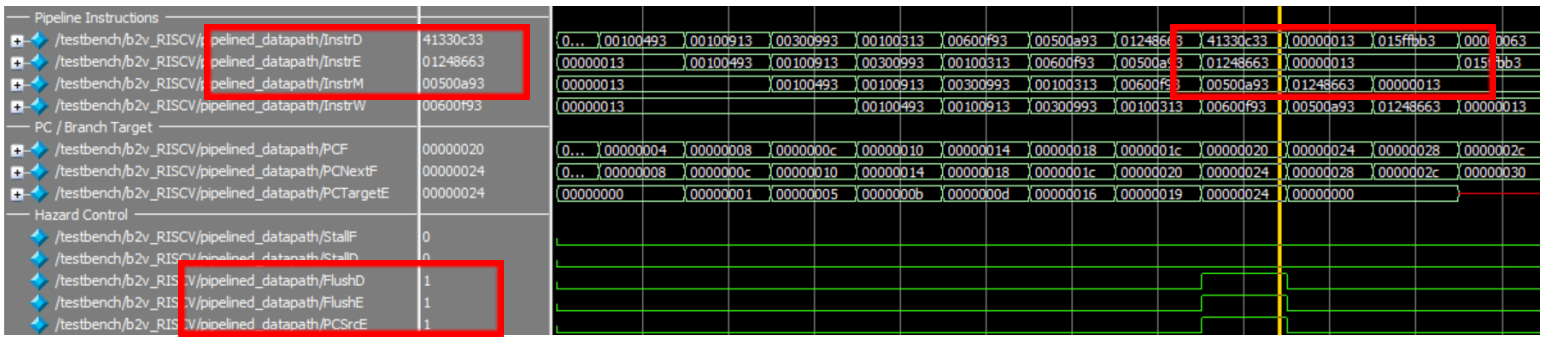
flush_test.s*

```
1  .section .text.main
2  .globl main
3
4  main:
5      addi s1, x0, 1       # x9  = 1
6      addi s2, x0, 1       # x18 = 1
7      addi s3, x0, 3       # x19 = 3
8      addi t1, x0, 1       # x6  = 1
9      addi t6, x0, 6       # x31 = 6
10     addi s5, x0, 5       # x21 = 5
11
12     beq  s1, s2, L1      # branch is taken because x9 == x18
13
14     sub  s8, t1, s3      # should be flushed
15     or   s9, t6, s5      # should be flushed
16
17 L1:
18     and  s7, t6, s5      # x23 = x31 AND x21 = 4
19
20 done:
21     beq x0, x0, done     # infinite loop
```

Flush Test Assembly Code

Code	Basic	Source
0x00100493	addi x9,x0,1	5: addi s1, x0, 1 # x9 = 1
0x00100913	addi x18,x0,1	6: addi s2, x0, 1 # x18 = 1
0x00300993	addi x19,x0,3	7: addi s3, x0, 3 # x19 = 3
0x00100313	addi x6,x0,1	8: addi t1, x0, 1 # x6 = 1
0x00600f93	addi x31,x0,6	9: addi t6, x0, 6 # x31 = 6
0x00500a93	addi x21,x0,5	10: addi s5, x0, 5 # x21 = 5
0x01248663	beq x9,x18,0x0000000c	12: beq s1, s2, L1 # branch is taken because x9 == x18
0x41330c33	sub x24,x6,x19	14: sub s8, t1, s3 # should be flushed
0x015fecb3	or x25,x31,x21	15: or s9, t6, s5 # should be flushed
0x015f7bb3	and x23,x31,x21	18: and s7, t6, s5 # x23 = x31 AND x21 = 4
0x00000063	beq x0,x0,0x00000000	21: beq x0, x0, done # infinite loop

In the waveform, the branch instruction is resolved in the Execute stage. Since the branch is taken, PCSrcE is asserted and the PC is updated to the branch target address. FlushD and FlushE are also asserted, which replaces the wrong instructions in the Decode and Execute stages with NOP instructions.

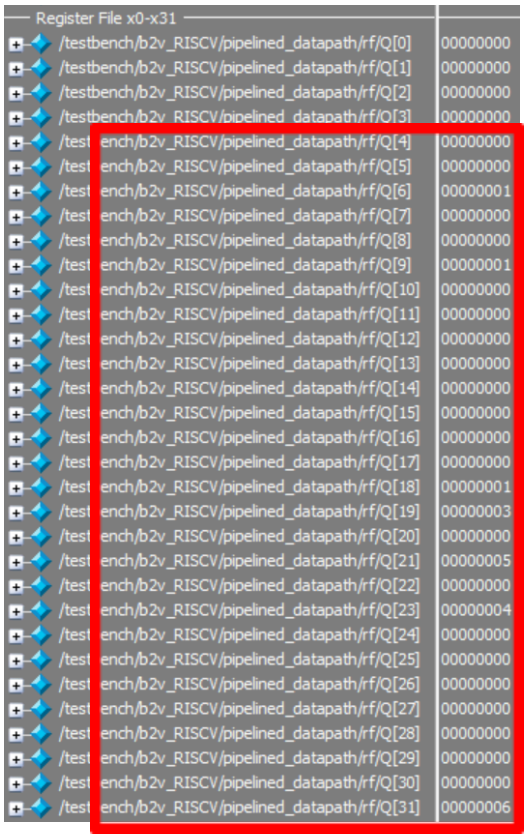


The final register values match the RARS simulation, showing that the flushed instructions did not incorrectly update the register file. This proves that the pipeline flush logic is working correctly.

Registers | Floating Point | Control and Status

Name	Number	Value
zero	0	0x00000000
ra	1	0x00000000
sp	2	0x00002ffc
gp	3	0x00001800
tp	4	0x00000000
t0	5	0x00000000
t1	6	0x00000001
t2	7	0x00000000
s0	8	0x00000000
s1	9	0x00000001
a0	10	0x00000000
a1	11	0x00000000
a2	12	0x00000000
a3	13	0x00000000
a4	14	0x00000000
a5	15	0x00000000
a6	16	0x00000000
a7	17	0x00000000
s2	18	0x00000001
s3	19	0x00000003
s4	20	0x00000000
s5	21	0x00000005
s6	22	0x00000000
s7	23	0x00000004
s8	24	0x00000000
s9	25	0x00000000
s10	26	0x00000000
s11	27	0x00000000
t3	28	0x00000000
t4	29	0x00000000
t5	30	0x00000000
t6	31	0x00000006

RARS Simulator Register Contents



Pipelined CPU Register Contents

Conclusion:

Seeing that the final register and RAM contents from the ModelSim simulation match the expected RARS results, it is evident that the pipelined CPU is functioning correctly. Furthermore, the waveform results also show that register forwarding, pipeline stalls, and pipeline flushes work as expected, proving that the CPU correctly handles the main hazards of a pipelined design.